

PROJECT WORK

LID DRIVEN CAVITY

Submitted by: -

**Kartik Gupta
(02086218)**

List of Figures

- 1 Computational Domain
- 2 Streamline patterns of cut-off case with (a) 16×16 , (b) 24×24 and (c) 32×32 grid size
- 3 Velocity profiles of with cut-off (with cavity) and without cut-off (without cavity) cases at Re 100 and 1000
- 4 Streamlines of cut-off case with (a) Re = 100, (b) Re = 400, (c) Re = 600, (d) Re = 800 and (e) Re = 1000 plotted using $t = 10s$
- 5 Streamlines of full cavity case with (a) Re = 100, (b) Re = 400, (c) Re = 600, (d) Re = 800 and (e) Re = 1000 plotted using $t = 10s$

1. Introduction

This research study deals with a square cavity, whose upper lid is moving with velocity “ v ”. This is a very popular single-phase problem in CFD which is generally used to test customized solvers. It involves the flow of an incompressible fluid inside the cavity driven by the tangential movement of the top wall, while the remaining walls are stationary (having no slip boundary condition).

2. Governing Equations

The governing equations for the problem are the Navier-Stokes equations, which describes the motion of the fluid. The conservation equations of mass and momentum are given by equation (1) and (2) respectively.

$$\nabla \cdot \vec{U} = 0 \quad (1)$$

$$\frac{\partial}{\partial t}(\rho \vec{U}) + \nabla \cdot (\rho \vec{U} \vec{U}) = -\nabla p + \nabla \cdot \tau + \vec{F}_B \quad (2)$$

Here, $\vec{U}$ is the velocity vector, ρ is the density and p is the pressure. $\vec{F}_B$ is the body force and τ represents the stress tensor given by,

$$\tau = \mu(\nabla \vec{U} + \nabla \vec{U}^T) \quad (3)$$

where μ is the coefficient of dynamic viscosity.

At the right, left and bottom walls of the domain a no-slip and no-penetration boundary condition is implemented as

$$\vec{U} \cdot \hat{n}_s = 0 \quad (4)$$

$$\vec{U} \cdot \hat{t}_s = 0 \quad (5)$$

3. Numerical Implementation

To solve the momentum equation (equation (2)), we use the two-step projection method of Chorin [1]. In this staggered arrangement the variables p and ρ are located at cell centers whereas velocities are defined at cell faces.

$$\frac{\rho^{n+1} \vec{U}^* - \rho^n \vec{U}^n}{\Delta t} = -\nabla \cdot (\rho \vec{U} \vec{U})^n + \nabla \cdot (\tau)^n + \vec{F}_B^n \quad (6)$$

$$\frac{\rho^{n+1} \vec{U}^{n+1} - \rho^{n+1} \vec{U}^*}{\Delta t} = -\nabla p^{n+1} \quad (7)$$

Here, n and $n+1$ denotes the current and next time-step respectively. $\vec{U}^*$ is the intermediate velocity. Applying the divergence free conditions in equation (7) we get pressure Poisson equation as follow:

$$\frac{\nabla^2 p^{n+1}}{\rho^{n+1}} = \frac{1}{\Delta t} \nabla \cdot \vec{U}^* \quad (8)$$

The pressure obtained from equation (8) is used to compute $\vec{U}^{n+1}$.

4. Computational Domain

The setup used in this study is a square cavity, whose left bottom side is restricted for the flow as shown in the figure 1. The length of the cavity is 1mm and the length of the cut-off is 0.25mm. This is a closed cavity of fluid with no new fluid entering. The fluid flow is driven by the moving upper lid of the cavity.

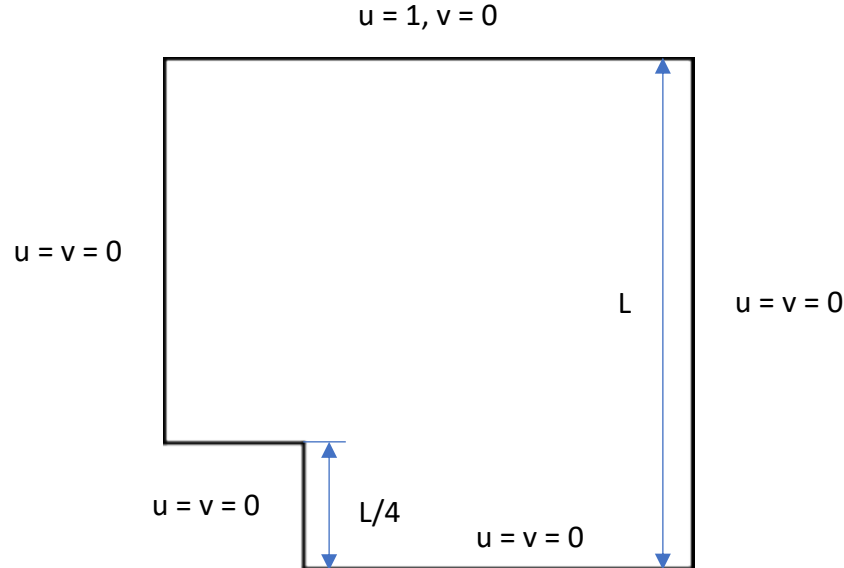


Figure 1: Computational Domain

5. Python Code

The code consists of 8 different files which are bc, constant, finalVel, input, interVel, press, run and simulation which are explained in detail as follows:

5.1 input

The file consists of the variables to be initialized in the simulation and the variables to be input as initial conditions for the simulation (like fluid properties, boundary conditions etc.).

```
import numpy as np
import matplotlib.pyplot as plot
import constant

# Domain Setup
# Length of domain
lx = constant.Geometry_in_x_direction
ly = constant.Geometry_in_y_direction
# No of cells
nx = constant.Cells_in_x_direction
ny = constant.Cells_in_y_direction
# Cell size
dx = lx/nx
dy = ly/ny
# Cell cavity
nxc = constant.Cavity_cells_in_x_direction
nyc = constant.Cavity_cells_in_y_direction

x = np.linspace(0, lx, nx)
y = np.linspace(0, ly, ny)
X, Y = np.meshgrid(x, y)

# Fluid properties
rho = constant.Density # Density
mu = constant.Viscosity # Viscosity
nu = mu/rho

# Initialize variables
u = np.zeros((ny+2,nx+2)) # Face centered
v = np.zeros((ny+2,nx+2)) # Face centered
p = np.zeros((ny+2,nx+2)) # Cell centered
div = np.zeros((ny+2,nx+2)) # Cell centered

u_bc = np.zeros((ny+2,nx+2)) # Initial velocities
v_bc = np.zeros((ny+2,nx+2))

initialVel = constant.Initial_Velocity # Velocity of lid
del_t = min ((0.25*dx*dy)/nu, 4*nu/initialVel**2)
twfin = 100 * del_t

# Boundary conditions
# Top wall
u_t = initialVel # Slip
v_t = 0.0 # Penetration
```

```

# Bottom wall
u_b = 0.0
v_b = 0.0
# Left wall
u_l = 0.0      # Penetration
v_l = 0.0      # Slip
# Right wall
u_r = 0.0
v_r = 0.0
# Cavity wall
u_c = 0.0
v_c = 0.0

myFile = open('pressureSolverConvergenceData.txt', 'w')
myFile.write('{:>8s} {:>8s} {:>8s}\n'.format('Timestep', 'Residual', '#-of-
iterations'))

```

5.2 bc

This file generates the specific boundary conditions for the lid driven cavity case and cuts-off a specific region from bottom left corner.

```

from input import *

def boundaryCondition(u, v, u_bc, v_bc, u_t, u_b, u_l, u_r, v_t, v_b, v_l, v_r, nx,
ny):
    # Setting initial velocities (BC to be applied on ghost cells)
    # Bottom wall
    u_bc[0, :] = 2*u_b - u_bc[1, :]
    v_bc[1, :] = v_b
    # Left wall
    u_bc[:, 1] = u_l
    v_bc[:, 0] = 2*v_l - v_bc[:, 1]
    # Right wall
    u_bc[:, nx+1] = u_r
    v_bc[:, nx+1] = 2*v_r - v_bc[:, nx]
    # Top wall
    u_bc[ny+1, :] = 2*u_t - u_bc[ny, :]
    v_bc[ny+1, :] = v_t
    # Cavity wall
    u_bc[0:nyc, 0:nxc+1] = u_c
    u_bc[nyc, 0:nxc] = 2*u_c - u_bc[nyc+1, 0:nxc]
    v_bc[0:nyc+1, 0:nxc] = v_c
    v_bc[0:nyc, nxc] = 2.0*v_c - v_bc[0:nyc, nxc+1]
    u[:] = u_bc[:]
    v[:] = v_bc[:]
    return u, v

```

5.3 interVel

After initializing the variables and setting up the specific boundary conditions the function in this file calculates the intermediate velocity over the entire domain.

```
from input import *

def intermediateVelocity(u, v, rho, mu, nx, ny, dx, dy, del_t):
    # Solving N-S equation for momentum control volume in x-axis
    for i in range (nxc+2, nx+1):
        for j in range (1, ny+1):
            u_e = 0.5*(u[j, i] + u[j, i+1])
            u_w = 0.5*(u[j, i-1] + u[j, i])
            u_n = 0.5*(u[j, i] + u[j+1, i])
            u_s = 0.5*(u[j, i] + u[j-1, i])
            v_n = 0.5*(v[j+1, i-1] + v[j+1, i])
            v_s = 0.5*(v[j, i-1] + v[j, i])
            convection = -((rho*u_e**2 - rho*u_w**2)/dx)-((rho*u_n*v_n -
rho*u_s*u_s)/dy)
            diffusion = mu*((u[j, i+1]-2*u[j, i]+u[j, i-1])/dx**2 + (u[j+1, i]-2*u[j,
i]+u[j-1, i])/dy**2)
            # calculating the intermediate velocity
            u[j, i] += (del_t*(convection+diffusion)/rho)
        for i in range (2, nxc+2):
            for j in range (nyc+1, ny+1):
                u_e = 0.5*(u[j, i] + u[j, i+1])
                u_w = 0.5*(u[j, i-1] + u[j, i])
                u_n = 0.5*(u[j, i] + u[j+1, i])
                u_s = 0.5*(u[j, i] + u[j-1, i])
                v_n = 0.5*(v[j+1, i-1] + v[j+1, i])
                v_s = 0.5*(v[j, i-1] + v[j, i])
                convection = -((rho*u_e**2 - rho*u_w**2)/dx)-((rho*u_n*v_n -
rho*u_s*u_s)/dy)
                diffusion = mu*((u[j, i+1]-2*u[j, i]+u[j, i-1])/dx**2 + (u[j+1, i]-2*u[j,
i]+u[j-1, i])/dy**2)
                # calculating the intermediate velocity
                u[j, i] += (del_t*(convection+diffusion)/rho)
    # Solving N-S equation for momentum control volume in y-axis
    for i in range (nxc+1, nx+1):
        for j in range (2, ny+1):
            v_e = 0.5*(v[j, i] + v[j, i+1])
            v_w = 0.5*(v[j, i] + v[j, i-1])
            v_n = 0.5*(v[j, i] + v[j+1, i])
            v_s = 0.5*(v[j, i] + v[j-1, i])
            u_e = 0.5*(u[j-1, i+1] + u[j, i+1])
            u_w = 0.5*(u[j-1, i] + u[j, i])
            convection = -((rho*u_e*v_e - rho*u_w*v_w)/dx)-((rho*v_n**2 -
rho*v_s**2)/dy)
```

```

        diffusion = mu*((v[j, i+1]-2*v[j, i]+v[j, i-1])/dx**2 + (v[j+1, i]-2*v[j,
i]+v[j-1, i])/dy**2)
        # calculating the intermediate velocity
        v[j, i] += (de_l_t*(convection+diffusion)/rho)
    for i in range(1, nxc+1):
        for j in range(nyc+2, ny+1):
            v_e = 0.5*(v[j, i] + v[j, i+1])
            v_w = 0.5*(v[j, i] + v[j, i-1])
            v_n = 0.5*(v[j, i] + v[j+1, i])
            v_s = 0.5*(v[j, i] + v[j-1, i])
            u_e = 0.5*(u[j-1, i+1] + u[j, i+1])
            u_w = 0.5*(u[j-1, i] + u[j, i])
            convection = -((rho*u_e*v_e - rho*u_w*v_w)/dx)-((rho*v_n**2 -
rho*v_s**2)/dy)
            diffusion = mu*((v[j, i+1]-2*v[j, i]+v[j, i-1])/dx**2 + (v[j+1, i]-2*v[j,
i]+v[j-1, i])/dy**2)
            # calculating the intermediate velocity
            v[j, i] += (de_l_t*(convection+diffusion)/rho)
    return u, v

```

5.4 press

This file generates the pressure field for calculating the velocity at next time step. It also uses an iterative solver for solving pressure field and hence is the bottleneck of the simulation.

```

from input import *
import constant

def pressureSolve(u, v, rho, nx, ny, dx, dy, de_l_t, t):
    # Pressure loop
    # Initializing variables and pressure coefficients
    p = np.zeros((ny+2, nx+2))
    rhs = np.zeros((ny+2, nx+2))
    a_p = np.zeros((ny+2, nx+2))
    a_e = np.ones((ny+2, nx+2))/(rho*dx**2)
    a_w = np.ones((ny+2, nx+2))/(rho*dx**2)
    a_n = np.ones((ny+2, nx+2))/(rho*dy**2)
    a_s = np.ones((ny+2, nx+2))/(rho*dy**2)
    # Pressure coefficients along domain boundary
    a_e[:, nx] = 0.0
    a_w[:, 1] = 0.0
    a_n[ny, :] = 0.0
    a_s[1, :] = 0.0
    a_s[nyc+1, 1:nxc] = 0.0
    a_w[1:nyc, nxc+1] = 0.0
    # Central pressure coefficient
    a_p = -(a_e + a_w + a_n + a_s)
    # Pressure solver tolerance

```



```

TOL = constant.Tolerance
beta = 1.872
counter = 0
maxError = 10
while (abs(maxError) > TOL):
    maxError = 0
    counter = counter + 1
    # Solve for pressure field
    for i in range(1, nx+1):
        for j in range(nyc+1, nx+1):
            rhs[j, i] = +(u[j, i+1] - u[j, i])/dx + (v[j+1, i] - v[j, i])/dy
            rhs[j, i] = rhs[j, i]/del_t - (a_w[j, i]*p[j, i-1] + a_e[j, i]*p[j,
i+1] + a_n[j, i]*p[j+1, i] + a_s[j, i]*p[j-1, i])
            p[j, i] = beta*rhs[j, i]/a_p[j, i] + (1-beta)*p[j, i]
        for i in range(nxc+1, nx+1):
            for j in range(1, nyc+1):
                rhs[j, i] = +(u[j, i+1] - u[j, i])/dx + (v[j+1, i] - v[j, i])/dy
                rhs[j, i] = rhs[j, i]/del_t - (a_w[j, i]*p[j, i-1] + a_e[j, i]*p[j,
i+1] + a_n[j, i]*p[j+1, i] + a_s[j, i]*p[j-1, i])
                p[j, i] = beta*rhs[j, i]/a_p[j, i] + (1-beta)*p[j, i]
            for i in range(1, nx+1):
                for j in range(nyc+1, nx+1):
                    rhs[j, i] = +(u[j, i+1] - u[j, i])/dx + (v[j+1, i] - v[j, i])/dy
                    error = a_w[j, i]*p[j, i-1] + a_e[j, i]*p[j, i+1] + a_n[j, i]*p[j+1,
i] + a_s[j, i]*p[j-1, i] + a_p[j, i]*p[j, i] - (rhs[j, i]/del_t)
                    if(np.abs(error) > maxError):
                        maxError = np.abs(error)
            for i in range(nxc+1, nx+1):
                for j in range(1, nyc+1):
                    rhs[j, i] = +(u[j, i+1] - u[j, i])/dx + (v[j+1, i] - v[j, i])/dy
                    error = a_w[j, i]*p[j, i-1] + a_e[j, i]*p[j, i+1] + a_n[j, i]*p[j+1,
i] + a_s[j, i]*p[j-1, i] + a_p[j, i]*p[j, i] - (rhs[j, i]/del_t)
                    if(np.abs(error) > maxError):
                        maxError = np.abs(error)
    myFile.write('{:>8s} {:>8s} {:>8s}\n'.format(str(t), str(maxError), str(counter)))
    return p

```

5.5 finalVel

The following code finally calculates the velocity at next time step using the pressure value from previous file.

```

from input import *
from press import *
from bc import *

def finalVelocity(p, u, v, rho, nx, ny, dx, dy, del_t):
    for i in range(nxc+2, nx+1):

```

```

    for j in range (1, ny+1):
        # Update U velocity
        u[j, i] -= del_t*(p[j, i] - p[j, i-1])/(dx*rho)
    for i in range (2, nxc+2):
        for j in range (nyc+1, ny+1):
            # Update U velocity
            u[j, i] -= del_t*(p[j, i] - p[j, i-1])/(dx*rho)
    for i in range (nxc+1, nx+1):
        for j in range (2, ny+1):
            # Update V velocity
            v[j, i] -= del_t*(p[j, i] - p[j-1, i])/(dy*rho)
    for i in range (1, nxc+1):
        for j in range (nyc+2, ny+1):
            # Update V velocity
            v[j, i] -= del_t*(p[j, i] - p[j-1, i])/(dy*rho)
    u_bc[:] = u[:]
    v_bc[:] = v[:]
    return u, v, u_bc, v_bc

```

5.6 simulation

The file below initializes the time loop for the simulation and call the functions which are described above for running a smooth computation.

```

from input import *
from bc import *
from interVel import *
from press import *
from finalVel import *
import constant

t = constant.Start_time # Start time for simulation
realTime = constant.End_time
iter = realTime/del_t
plt = 0
pldt = 1
while t < iter:
    u, v = boundaryCondition(u, v, u_bc, v_bc, u_t, u_b, u_l, u_r, v_t, v_b, v_l, v_r,
nx, ny) # Set boundary conditions
    u, v = intermediateVelocity(u, v, rho, mu, nx, ny, dx, dy, del_t) #Solve for
intermediate velocity condition
    p = pressureSolve(u, v, rho, nx, ny, dx, dy, del_t, iter) #Pressure iterative
solver
    u, v, u_bc, v_bc = finalVelocity(p, u, v, rho, nx, ny, dx, dy, del_t) #Advance
time step

    plt = plt + 1
    t = t + 1

```

```

    realTime = realTime + del_t

U = np.zeros((ny, nx))
V = np.zeros((ny, nx))
velMag = np.zeros((ny, nx))

# Interpolate my cell centered U and V velocites
for i in range(nxc+1, nx):
    for j in range(1, ny):
        U[j, i] = 0.5 * (u[j+1, i+1] + u[j+1, i+2])
        V[j, i] = 0.5 * (v[j+1, i+1] + v[j+2, i+1])
        velMag[j, i] = np.sqrt(U[j, i]**2 + V[j, i]**2)
for i in range(1, nxc):
    for j in range(nyc+1, ny):
        U[j, i] = 0.5 * (u[j+1, i+1] + u[j+1, i+2])
        V[j, i] = 0.5 * (v[j+1, i+1] + v[j+2, i+1])
        velMag[j, i] = np.sqrt(U[j, i]**2 + V[j, i]**2)

myFile.close()

# Plot stream particles
fig1, ax1 = plot.subplots(figsize=(6, 6))
strm = ax1.streamplot(x, y, U, V, color=U, linewidth=2, cmap=plot.cm.autumn)
cbar = plot.colorbar(strm.lines)
cbar.ax.set_ylabel('Velocity magnitude')
ax1.set_title('Stream Particles')
ax1.set_xlabel('X-coordinate (m)')
ax1.set_ylabel('Y-coordinate (m)')

# Plot streamlines
#fig3, ax3 = plot.subplots()
#startX, startY = np.meshgrid(np.arange(0, lx, 0.2), np.arange(0, ly, 0.3))
#strm = ax3.streamplot(x, y, U, V, start_points=np.array([startX, startY]).T,
#linewidth=1.5, arrowsize=1.5)
#ax3.set_title('Velocity Streamline')
#ax3.set_xlabel('X-coordinate (m)')
#ax3.set_ylabel('Y-coordinate (m)')

# Plot velocity vectors
fig2, ax2 = plot.subplots()
q = ax2.quiver(x, y, U, V, units='width', color='k', scale=2)
ax2.set_title('Velocity Vector')
ax2.set_xlabel('X-coordinate (m)')
ax2.set_ylabel('Y-coordinate (m)')

```

```
plot.show()
```

5.7 constant

constant file contains the initial values given to the simulation for performing the computational study.

```
# Length of domain
Geometry_in_x_direction = 1.0
Geometry_in_y_direction = 1.0

# No of cells
Cells_in_x_direction = 16
Cells_in_y_direction = 16

# Cell cavity
Cavity_cells_in_x_direction = 4
Cavity_cells_in_y_direction = 4

# Fluid properties
Density = 1.0
Viscosity = 0.01

# Boundary condition
Initial_Velocity = 1.0

# Pressure
Tolerance = 1e-2

# Time
Start_time = 0
End_time = 1
```

5.8 run

This file is used to run the above described customized solver.

```
#!/bin/bash
```

6. Results

6.1 Mesh Study

We perform three different simulations using 16×16 , 24×24 and 32×32 mesh grid. The simulations are run with $Re = 100$ and $t = 10s$ as shown in figure 2.

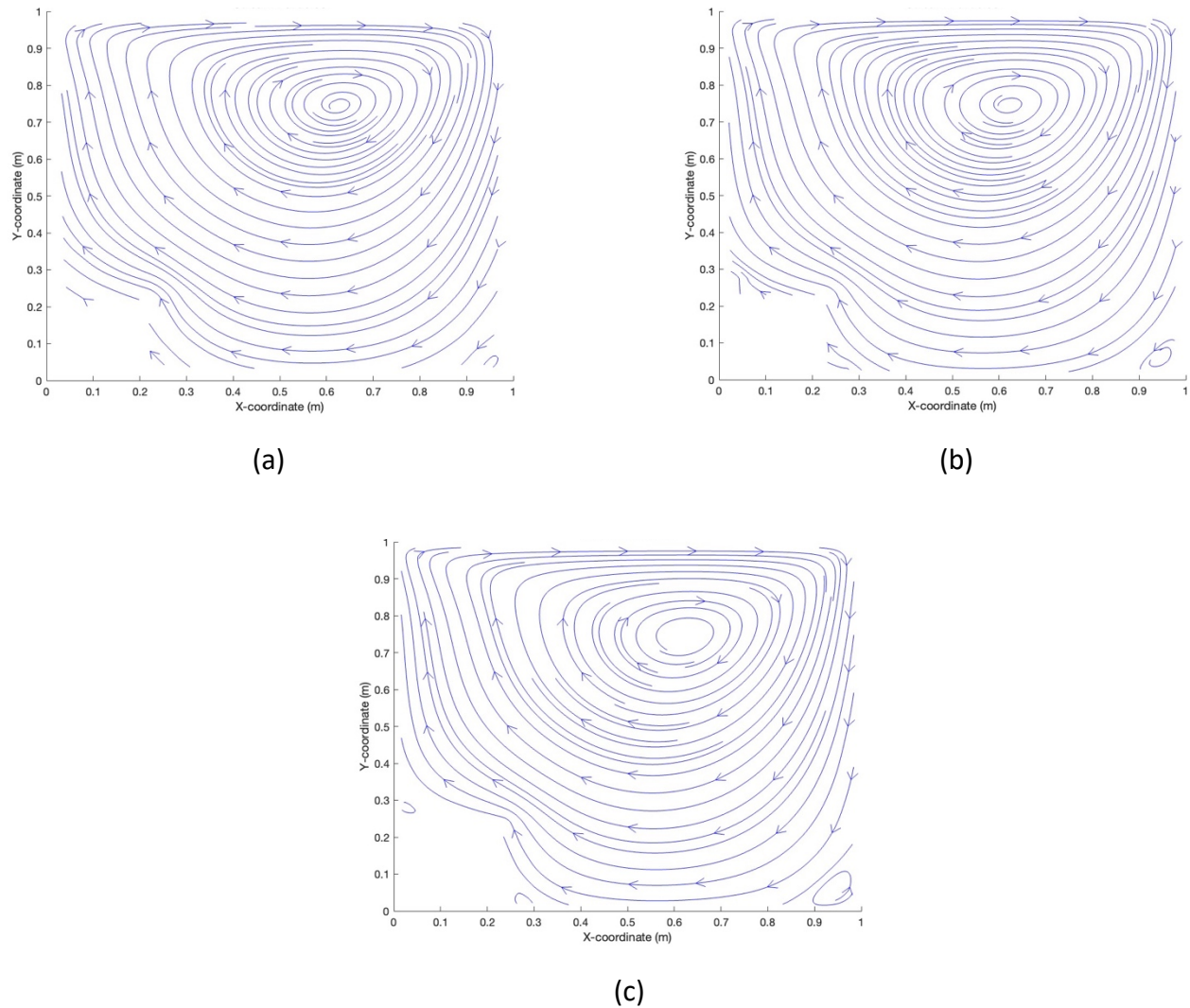


Figure 2: Streamline patterns of cut-off case with (a) 16×16 , (b) 24×24 and (c) 32×32 grid size.

The fluid inside the cavity energizes due to the top wall of the lid and insert tangential velocity into the steady fluid. This pushes the fluid on the right wall and allows it to move downward till it reaches the left

side of the cavity making a large symmetric vortex centered at top region of the cavity (top-right corner). At low Re , the velocities at the center of the cavity are less due to the dissipation of the energy through large viscous term and flow being laminar. As we increase the Re to 100 the flow switches to the transition mode and the large vortex becomes non symmetric and small vortex at bottom right corner appears which makes the flow more chaotic.

As we increase the grid cells, refinement level enhances, and we can see vortices at bottom right corner being developed properly. Finer resolution leads to highly accurate results with a compromise in high computational time. Therefore, selecting an appropriate mesh is very tedious task as well as mandatory for deciding the faith of the simulation. In this study we choose 24×24 grid for performing further simulations.

6.2 Validation of Test Case

The first and the most important step after compiling flow solver is the validation of the results with different research studies. In this validation we have compared the velocity results of our code with the results obtained by Ghia et. al [2].

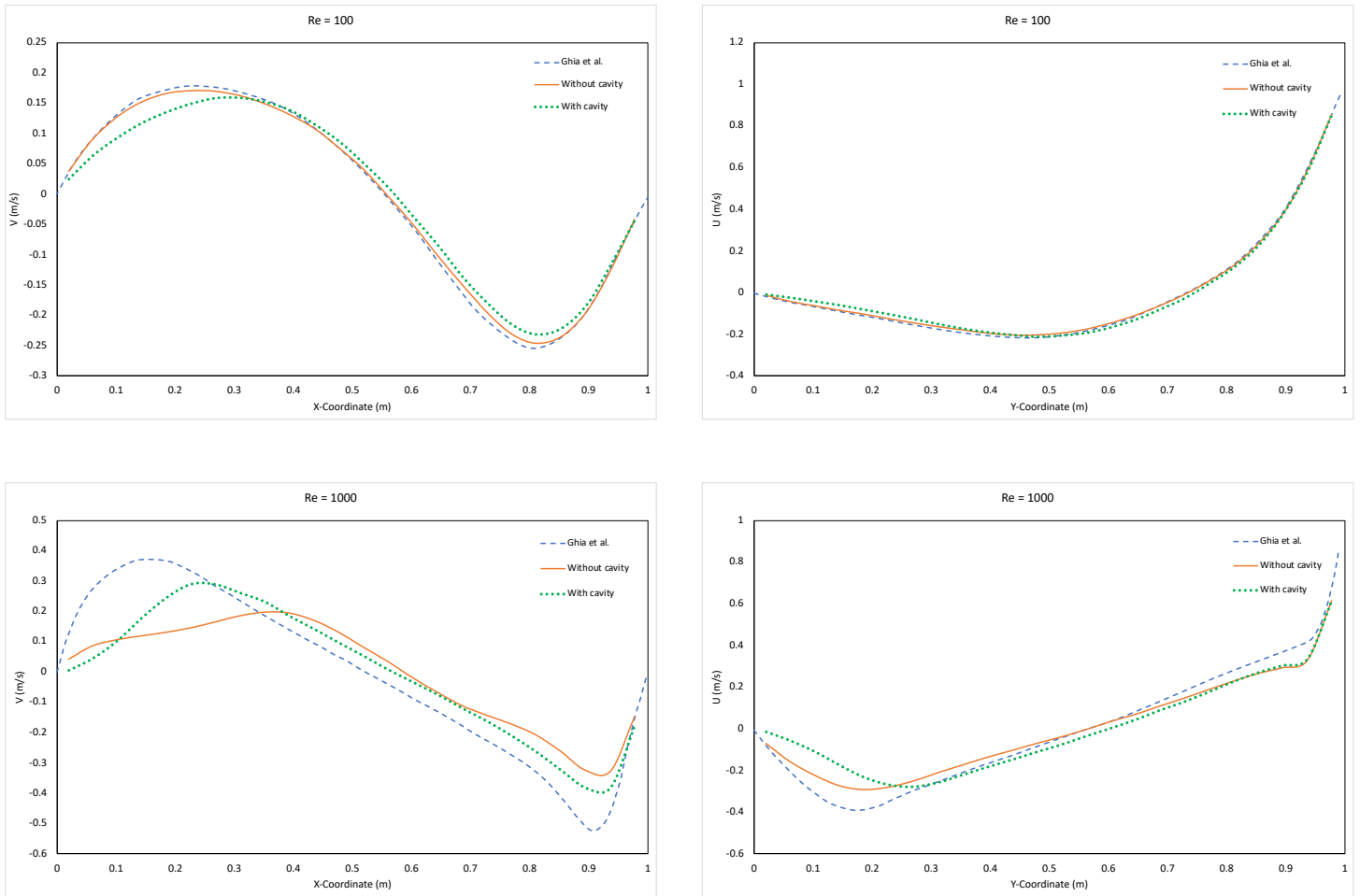
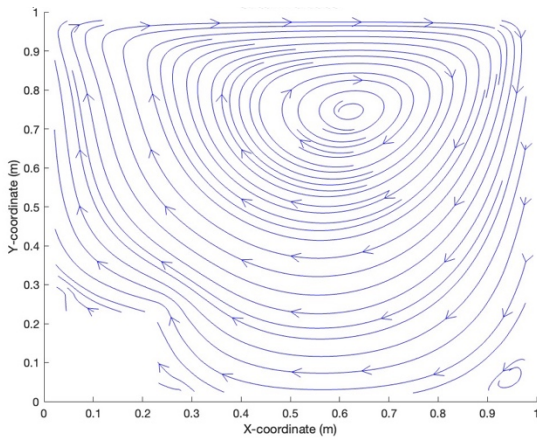


Figure 3: Velocity profiles of with cut-off (with cavity) and without cut-off (without cavity) cases at Re 100 and 1000.

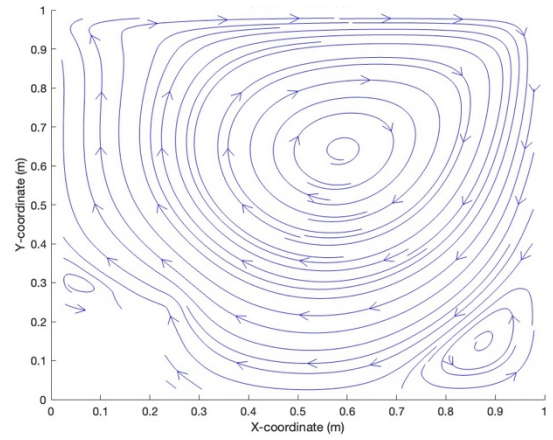
Figure 3, clearly depicts that the results of the full cavity case and the cavity case with cut-off is in good agreement with the results of Ghia et al. for low Re, but as Re increases, we can observe small deviation in the results from Ghia et al.

Also, there is significant difference in the velocity curves of with and without cut-off cases, which increases with increase in Reynolds number. This is probably due to the hindrance created by cut-off, which results in enhanced chaotic and turbulent flow.

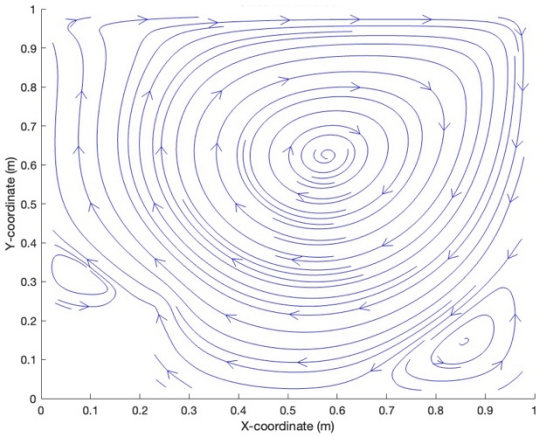
6.3 Variation in Reynolds Number



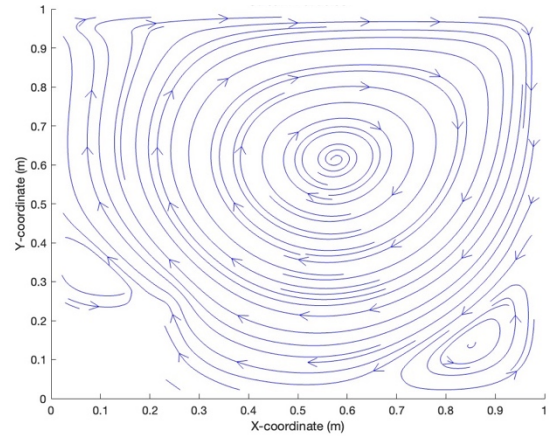
(a)



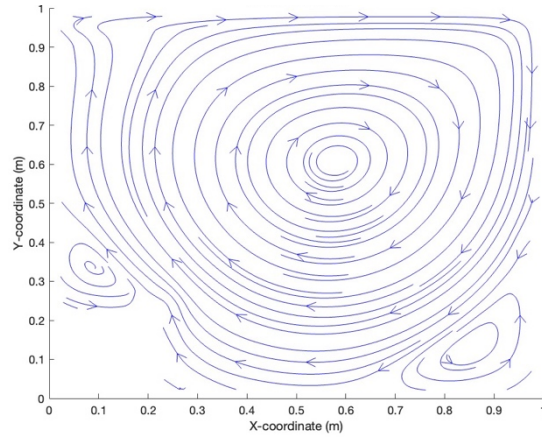
(b)



(c)



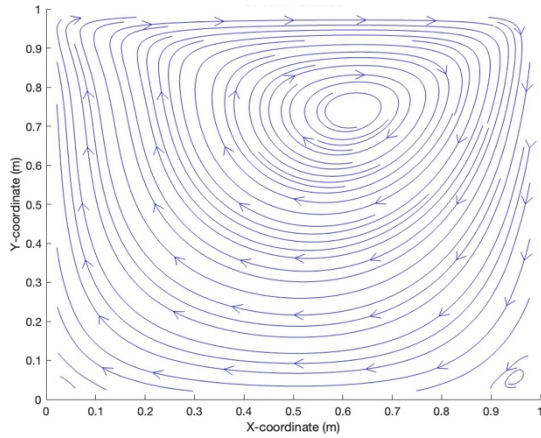
(d)



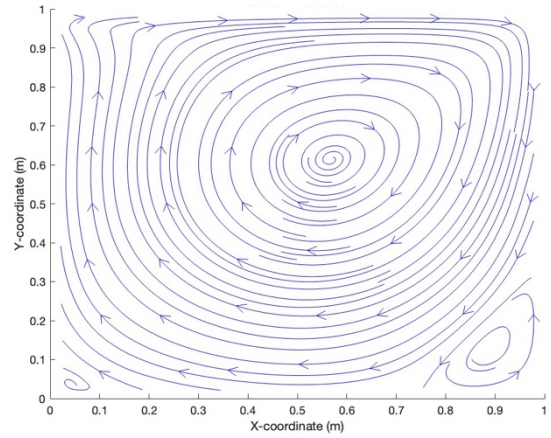
(e)

Figure 4: Streamlines of cut-off case with (a) $Re = 100$, (b) $Re = 400$, (c) $Re = 600$, (d) $Re = 800$ and (e) $Re = 1000$ plotted using $t = 10s$.

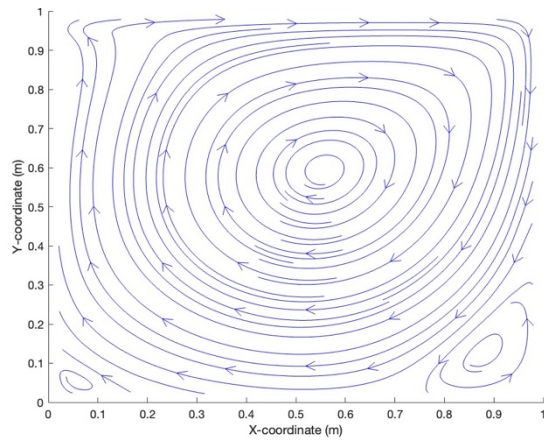
As we increase the Reynolds number further from 100 to 1000, we can see more chaotic flow with the primary vortex being shifted to the center of the cavity due to flow inertia. With increase in Re , the flow starts separating from the walls creating more corner vortices as depicted in figure 4. In figure 4(a), there is no vortex at the left cut-off area, but as we increase the Re , corner vortex starts growing bigger and bigger.



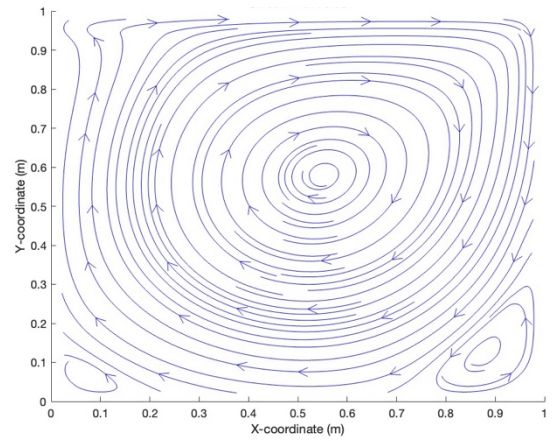
(a)



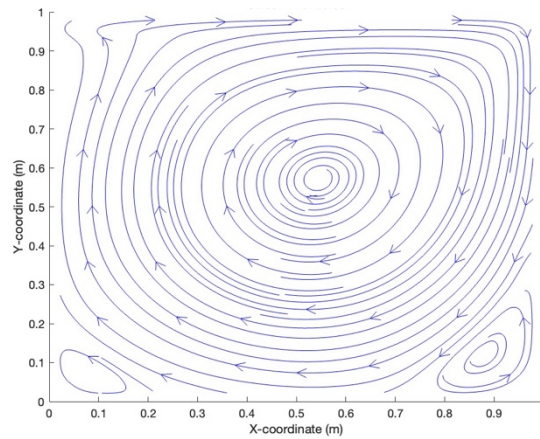
(b)



(c)



(d)



(e)

Figure 5: Streamlines of full cavity case with (a) $Re = 100$, (b) $Re = 400$, (c) $Re = 600$, (d) $Re = 800$ and (e) $Re = 1000$ plotted using $t = 10s$.

Similarly, on comparing the cut-off case with the full cavity case we can easily depict that as we increase the Re , the cut-off region applies more hindrance to the flow and due to the flow inertia, the fluid starts separating and vortices are formed. This hindrance makes the flow more chaotic that even at low Re we can observe more developed vortices. This is evident as we compare the results of $Re = 1000$. On comparing both the figures we see a fully developed vortex on bottom right corner, but as we investigate the left corner, we analyze that in case of hinderance the vortex is fully developed whereas, the vortex is still forming on the other case.

7. Conclusion

The code works fine for the classic CFD problem, the lid driven cavity. The study is being conducted with multiple Reynolds number and a detailed comparison is done for the cases with and without cut-off region. With the increase in Reynolds number, flow becomes more chaotic and asymmetric, which results in more vortices being developed (shredded from primary vortex). The cut-off region is also responsible for the increase in turbulence of the flow. In addition, we have compared the results to existing literature and found a good agreement between the results.

8. References

- [1] C.Hoi, "flowSolver_teachingModule," Matlab notes.
- [2] U. Ghia, K. N. Ghia, and C. T. Shin, "High-Re solutions for incompressible flow using the Navier-Stokes equations and a multigrid method," *J. Comput. Phys.*, vol. 48, no. 3, pp. 387–411, Dec. 1982, doi: 10.1016/0021-9991(82)90058-4.